Chương 2 (Phần 2): Lập trình C an toàn – Quản lý bộ nhớ

2.4 Sắp xếp cấu trúc dữ liệu – 2.5 Công cụ phân tích tĩnh – 2.6 Tổng kết

ThS. Vũ Quang Dũng

Khoa Hệ thống thông tin – Trường Đại học Phenikaa

CSE703093 – An toàn phần mềm

Nhắc lại Phần 1

- 2.1 Đánh giá rủi ro: attack surface, likelihood/impact cho mã C
- 2.2 Cấu trúc dữ liệu động: con trỏ, heap, linked list, ownership
- 2.3 Lập trình C an toàn: hàm an toàn, UB, sanitizer, checklist review

Phần 2 sẽ hoàn thiện với: alignment, công cụ phân tích tĩnh, và tổng kết chương.

Nội dung Phần 2

1. 2.4 Sắp xếp cấu trúc dữ liệu (Data Structure Alignment)
2. 2.5 Công cụ phân tích tĩnh tìm lỗi bảo mật
3. 2.6 Tổng kết Chương 2

Vì sao CPU yêu cầu dữ liệu được căn chỉnh (aligned)?

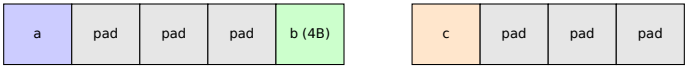
- CPU đọc/ghi bộ nhớ theo *từ* (word) có kích thước cố định (4 byte, 8 byte...), bắt đầu tại địa chỉ chia hết cho kích thước đó
- Truy cập dữ liệu **không căn chỉnh** (misaligned) trên một số kiến trúc có thể: chậm hơn (cần 2 lần đọc bộ nhớ), hoặc gây lỗi phần cứng (bus error) trên kiến trúc nghiêm ngặt (vd: một số CPU ARM/SPARC)
- Vì vậy trình biên dịch C tự động thêm **padding** (đệm) vào struct để đảm bảo alignment

Ví dụ Padding trong Struct

```c
struct Example {
    char a;      // 1 byte
    int b;       // 4 byte (can chinh: dia chi chia het cho 4)
    char c;      // 1 byte
};
printf("%zu\n", sizeof(struct Example));  // Ket qua: 12, khong phai 6!
```

Trình biên dịch chèn 3 byte đệm sau a (để b bắt đầu tại địa chỉ chia hết cho 4) và 3 byte đệm sau c (để struct có kích thước là bội số của alignment lớn nhất bên trong).

Minh họa bố cục bộ nhớ của struct



Tổng cộng: 1 (a) + 3 (pad) + 4 (b) + 1 (c) + 3 (pad) = 12 byte.

Quy tắc căn chỉnh cơ bản

- Mỗi kiểu dữ liệu có *alignment requirement* riêng: `char` = 1, `short` = 2, `int`/`float` = 4, `double`/`long long` = 8 (thường gặp trên x86-64)
- Địa chỉ bắt đầu của mỗi trường phải chia hết cho alignment của kiểu đó
- Kích thước tổng của struct phải là bội số của alignment *lớn nhất* trong các trường (để mảng struct vẫn giữ đúng alignment cho mọi phần tử)

Sắp xếp lại trường để giảm Padding

```
// Khong toi uu: 12 byte
struct Bad { char a; int b; char c; };

// Toi uu: sap xep tu lon den nho -> chi 8 byte
struct Good { int b; char a; char c; };
```

Bằng cách nhóm các trường có cùng yêu cầu alignment lại gần nhau (thường sắp từ kiểu lớn nhất đến nhỏ nhất), có thể giảm đáng kể padding lãng phí – quan trọng khi có hàng triệu instance (vd: struct dùng trong mảng lớn hoặc packet mạng).

Bài tập: Tính kích thước Struct

Cho kiến trúc x86-64 (alignment: char=1, short=2, int=4, double=8):

```
struct Data {
    char flag;
    double value;
    short id;
};
```

Hãy tính `sizeof(struct Data)`.

Đáp án: Tính kích thước Struct

- `flag` (1 byte) tại offset 0
- Padding 7 byte để `value` (8 byte, cần alignment 8) bắt đầu tại offset 8
- `value` chiếm offset 8–15
- `id` (2 byte) tại offset 16–17
- Padding 6 byte để tổng kích thước là bội số của 8 (alignment lớn nhất) $\Rightarrow$ tổng = 24 byte

#pragma pack và __attribute__((packed))

```
#pragma pack(push, 1)
struct PackedData {
    char flag;
    double value;
    short id;
};
#pragma pack(pop)
// sizeof(PackedData) = 11 (khong padding)
```

Lưu ý

Loại bỏ padding giúp tiết kiệm bộ nhớ (quan trọng cho packet mạng, file format nhị phân) nhưng có thể làm **chậm truy cập** (misaligned access) và trên một số kiến trúc có thể gây lỗi runtime.

Alignment và bảo mật: Rò rỉ thông tin qua Padding

Định nghĩa

Padding byte không được khởi tạo có thể chứa dữ liệu ngẫu nhiên còn sót lại trong bộ nhớ (từ lần cấp phát trước) – nếu struct được ghi trực tiếp ra file/mạng (vd: qua `memcpy` hoặc gửi qua socket), padding có thể **rò rỉ dữ liệu nhạy cảm** từ bộ nhớ cũ.

Ví dụ

Lỗ hổng thực tế dạng này từng được phát hiện trong các giao thức mạng truyền trực tiếp struct C ra ngoài mà không dùng `memset` khởi tạo toàn bộ vùng nhớ trước.

Phòng ngừa rò rỉ qua Padding

```
struct Packet pkt;
memset(&pkt, 0, sizeof(pkt));   // Khoi tao TOAN BO vung nho, ke ca padding
pkt.flag = 1;
pkt.value = 3.14;
send(sock, &pkt, sizeof(pkt), 0);   // An toan: khong con du lieu rac
```

Luôn `memset` toàn bộ struct về 0 trước khi điền dữ liệu, đặc biệt khi struct sẽ được truyền ra ngoài chương trình (file, mạng).

Alignment và Cache Line (giới thiệu hiệu năng)

- CPU hiện đại đọc bộ nhớ theo *cache line* (thường 64 byte)
- Struct được căn chỉnh tốt, kích thước phù hợp cache line giúp giảm số lần truy cập bộ nhớ chậm
- **False sharing**: khi 2 biến của 2 luồng khác nhau nằm trên cùng cache line, hiệu năng giảm mạnh dù không có xung đột dữ liệu thực sự – vấn đề hiệu năng nằm ngoài phạm vi bảo mật nhưng liên quan tới cùng nguyên lý alignment

Kiểm tra alignment bằng công cụ

```
# Xem offset cua tung truong trong struct (GDB)
gdb -batch -ex "ptype /o struct Example" ./program

# pahole: cong cu chuyen dung phan tich padding struct
pahole ./program
```

Công cụ `pahole` (Poke-a-Hole) đặc biệt hữu ích để tìm và tối ưu các struct có nhiều padding lãng phí trong dự án C lớn (vd: nhân Linux).

Bài tập: Sắp xếp lại Struct tối ưu

```
struct Mixed {
    char a;
    long long b;
    char c;
    int d;
    char e;
};
```

Hãy sắp xếp lại thứ tự các trường để giảm thiểu padding (giả sử alignment: char=1, int=4, long long=8).

Đáp án: Sắp xếp lại Struct tối ưu

```
struct MixedOptimized {
    long long b;   // 8 byte
    int d;         // 4 byte
    char a;        // 1 byte
    char c;        // 1 byte
    char e;        // 1 byte
                   // 1 byte padding con lai (bicanh 8)
};
```

Từ 32 byte (bản gốc, nhiều padding xen kẽ) giảm xuống còn 16 byte – tiết kiệm 50% bộ nhớ cho mỗi instance.

Liên hệ Alignment với Chương 3: Memory Model

Chương 3 (mục 3.6, 3.7) sẽ trình bày **Memory Model** và **Aligned Memory Model** một cách hình thức hơn – dùng để *chứng minh* tính đúng đắn của các phép biến đổi bộ nhớ (bao gồm alignment) trong các công cụ kiểm chứng tự động.

Union và Alignment

```
union Value {
    char c;
    int i;
    double d;
};
printf("%zu\n", sizeof(union Value));  // = 8 (kich thuoc truong lon nhat)
```

- union chỉ cấp phát đủ chỗ cho trường *lớn nhất*, và alignment của union bằng alignment lớn nhất trong các trường
- Rủi ro: tất cả các trường *chia sẻ cùng vùng nhớ* – ghi vào trường này rồi đọc từ trường khác (type punning) là hành vi dễ gây lỗi/UB nếu không cẩn thận

Bit-field trong Struct (giới thiệu)

```c
struct Flags {
    unsigned int is_active : 1;
    unsigned int is_admin  : 1;
    unsigned int level     : 4;
};
```

Bit-field cho phép tiết kiệm bộ nhớ ở mức bit, nhưng thứ tự sắp xếp bit-field trong bộ nhớ **phụ thuộc trình biên dịch/kiến trúc** (không được chuẩn C quy định chặt chẽ) – cần cẩn trọng khi dùng trong giao thức mạng/file format cần tính di động (portability).

Từ khóa _Alignas và _Alignof (C11)

```c
#include <stdalign.h>
_Alignas(16) char buffer[64];   // Ep buon can chinh 16 byte
printf("%zu\n", _Alignof(int)); // In ra alignment yeu cau cua int
```

C11 chuẩn hóa cách kiểm soát alignment tường minh, hữu ích khi làm việc với phần cứng đặc biệt (SIMD instructions thường yêu cầu alignment 16/32 byte) hoặc khi cần tối ưu cache.

False Sharing – Ví dụ minh họa (đa luồng)

```
struct Counters {
    int counter_thread1;  // Cung cache line voi counter_thread2!
    int counter_thread2;
};
```

Nếu 2 luồng khác nhau liên tục ghi vào `counter_thread1` và `counter_thread2` nằm trên *cùng một cache line*, CPU phải đồng bộ cache liên tục giữa các lõi dù dữ liệu logic không xung đột – gây giảm hiệu năng nghiêm trọng dù không phải lỗi bảo mật.

Giải pháp False Sharing: Padding thủ công

```
struct Counters {
    int counter_thread1;
    char padding[60];   // Dam bao khac cache line (64 byte)
    int counter_thread2;
};
```

Ở đây, lập trình viên *chủ động* thêm padding (ngược lại với việc cố loại bỏ padding ở phần trước) để tách 2 biến ra 2 cache line riêng biệt – minh họa alignment có thể được điều chỉnh cho *cả hai mục tiêu*: tiết kiệm bộ nhớ hoặc tối ưu hiệu năng đa luồng.

Endianness – Khái niệm liên quan

Định nghĩa

Endianness: thứ tự byte được lưu trữ trong bộ nhớ cho kiểu dữ liệu nhiều byte (vd: `int` 4 byte) – *little-endian* (byte thấp trước, x86) hay *big-endian* (byte cao trước, một số kiến trúc mạng/mainframe).

Cùng với alignment, endianness là một nguồn lỗi phổ biến khi truyền dữ liệu nhị phân (struct) giữa các hệ thống khác kiến trúc – một lý do khác để cẩn trọng khi ghi struct trực tiếp ra file/mạng.

Bài tập: Mảng Struct và Alignment

Cho `struct Point { char tag; int x, y; }` và một mảng `Point points[1000]`. Alignment của struct ảnh hưởng gì tới việc truy cập `points[i]` nhanh hay chậm?

Đáp án: Mảng Struct và Alignment

- Kích thước mỗi struct phải là bội số của alignment lớn nhất (ở đây là 4, do `int`) để *mọi* phần tử trong mảng đều được căn chỉnh đúng, không chỉ phần tử đầu tiên
- Nếu không có padding cuối struct, `points[1]` sẽ bắt đầu tại địa chỉ không chia hết cho 4 $\Rightarrow$ truy cập chậm hoặc lỗi trên một số kiến trúc
- Đây chính là lý do struct luôn có kích thước là bội số của alignment lớn nhất, dù trường cuối không cần padding khi đứng riêng lẻ

Alignment trong ngôn ngữ khác: So sánh nhanh

- Python, Java: không cần lo alignment thủ công – runtime/VM tự quản lý bố cục bộ nhớ đối tượng
- Rust: có `#[repr(C)]` để kiểm soát bố cục tương thích C khi cần, mặc định trình biên dịch tự tối ưu bố cục
- C/C++: lập trình viên có toàn quyền kiểm soát (và toàn bộ trách nhiệm) về alignment và padding

Bài tập: Alignment trong Protocol Design

Khi thiết kế một giao thức nhị phân (binary protocol) để hai hệ thống C giao tiếp qua mạng, có nên dựa vào bố cục struct mặc định của trình biên dịch (có padding tự động) hay không? Giải thích.

Đáp án: Alignment trong Protocol Design

- **Không nên** dựa vào bố cục mặc định – padding và alignment có thể khác nhau giữa các trình biên dịch/kiến trúc (32-bit vs 64-bit, x86 vs ARM)
- Thực hành đúng: định nghĩa tường minh từng byte của giao thức (dùng `#pragma pack(1)` hoặc serialize/deserialize thủ công từng trường), không truyền trực tiếp `struct` qua mạng
- Đây cũng giải quyết luôn vấn đề endianness đã đề cập

Tổng kết 2.4

- Alignment đảm bảo CPU truy cập bộ nhớ hiệu quả và đúng đắn
- Padding tự động được trình biên dịch chèn vào, có thể tối ưu bằng cách sắp xếp lại trường
- Padding chưa khởi tạo có thể là lỗ hổng *rò rỉ thông tin* nếu struct được truyền ra ngoài
- Luôn `memset` struct trước khi gửi qua mạng/ghi ra file

Nhắc lại Static Analysis (từ Chương 1)

Định nghĩa

Static Analysis: phân tích mã nguồn/mã máy để phát hiện lỗi mà *không cần thực thi* chương trình.

Chương này áp dụng cụ thể vào mã C: các công cụ thực tế, cách đọc kết quả, và giới hạn (false positive/negative).

Phân loại kỹ thuật Static Analysis

Pattern-based	Tìm kiếm mẫu mã nguy hiểm đã biết (vd: lời gọi `strcpy`) – nhanh nhưng đơn giản, dễ bỏ sót
Data-flow analysis	Theo dõi luồng giá trị biến qua chương trình (vd: taint analysis đã học ở 1.4)
Abstract Interpretation	Tính toán một *xấp xỉ an toàn* của mọi trạng thái có thể có của chương trình – nền tảng lý thuyết mạnh nhất, cơ sở cho các công cụ công nghiệp như Astrée

Công cụ: cppcheck

```
cppcheck --enable=all --inconclusive vulnerable.c
```

- Công cụ static analysis miễn phí, phổ biến cho C/C++
- Phát hiện: buffer overflow tiềm ẩn, null pointer dereference, memory leak, biến không dùng
- Không cần biên dịch được mã nguồn (phân tích trực tiếp), phù hợp phân tích nhanh

Ví dụ output của cppcheck

```
$ cppcheck vulnerable.c
[vulnerable.c:5]: (error) Buffer overflow: buffer
[vulnerable.c:8]: (warning) Possible null pointer
    dereference: result
[vulnerable.c:12]: (style) Variable 'unused' is
    assigned a value that is never used
```

Kết quả phân loại theo mức độ: `error` (khả năng cao là lỗi thật), `warning`, `style` (vấn đề chất lượng mã, không nhất thiết là lỗ hổng).

Công cụ: Clang Static Analyzer

```
scan-build gcc -c vulnerable.c
```

- Dùng kỹ thuật *symbolic execution* (đã giới thiệu ở Chương 1) để mô phỏng các đường thực thi có thể có
- Phát hiện lỗi sâu hơn cppcheck: use-after-free, null dereference qua nhiều bước gọi hàm, memory leak phức tạp
- Xuất báo cáo HTML trực quan, chỉ ra từng bước dẫn tới lỗi (path-sensitive)

Công cụ: clang-tidy

```
clang-tidy vulnerable.c -- -std=c11
```

- Tập trung vào việc tuân thủ coding standard (CERT C, các quy tắc hiện đại hóa mã nguồn)
- Có thể tự động sửa một số lỗi đơn giản (`-fix`)
- Thường tích hợp trực tiếp vào IDE (VS Code, CLion) để cảnh báo ngay khi gõ mã

False Positive và False Negative trong Static Analysis

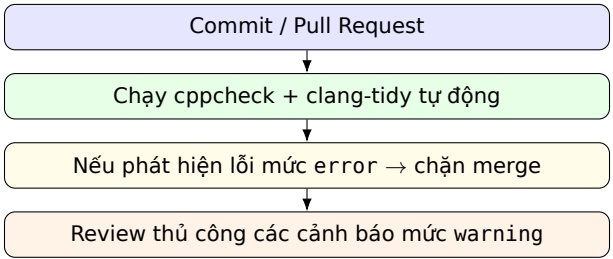
False Positive	Công cụ báo có lỗi nhưng thực tế mã nguồn đúng (gây phiền toái, tốn thời gian rà soát)	Liên hệ khái niệm
False Negative	Công cụ *không* báo lỗi dù thực tế có lỗ hổng (nguy hiểm hơn vì tạo cảm giác an toàn giả)	

sound và *complete* đã học ở Chương 1: một công cụ static analysis lý tưởng vừa sound (không false negative) vừa complete (không false positive) – nhưng thực tế luôn phải đánh đổi.

Đánh đổi Sound vs. Practical trong công cụ thực tế

- Công cụ **sound tuyệt đối** (không bỏ sót lỗi nào) thường tạo *quá nhiều* false positive, khiến không thực dụng
- Công cụ thương mại/mã nguồn mở thường chọn cân bằng: chấp nhận một số false negative để giảm nhiễu, giúp lập trình viên thực sự dùng được
- Đây là lý do cần **kết hợp nhiều kỹ thuật** (static + dynamic + fuzzing) thay vì chỉ dựa vào một công cụ duy nhất

Tích hợp Static Analysis vào CI/CD



Tự động hóa giúp phát hiện lỗi *ngay khi* mã được viết ra, thay vì để tới giai đoạn kiểm thử/vận hành (liên hệ biểu đồ chi phí sửa lỗi ở Chương 1).

Thực hành: Chạy cppcheck trên đoạn mã đã học

```
void copy_input(char *user_input) {
    char buffer[64];
    strcpy(buffer, user_input);
}
```

```
$ cppcheck copy_input.c
[copy_input.c:3]: (warning) Either the condition
  'user_input==0' is redundant or there is possible
  null pointer dereference: user_input.
```

Lưu ý: cppcheck ở đây chỉ cảnh báo null pointer, *không* phát hiện được nguy cơ buffer overflow của `strcpy` trong ví dụ đơn giản này – minh họa rõ vấn đề false negative.

Taint Analysis Tool – Ví dụ khái niệm

```
char cmd[256];
scanf("%s", cmd);         // source: du lieu tu nguoi dung (tainted)
system(cmd);               // sink: thuc thi lenh he thong
```

Công cụ taint analysis chuyên dụng sẽ đánh dấu `cmd` là *tainted* ngay từ `scanf`, theo dõi nó không qua bước ”làm sạch” nào, và cảnh báo khi nó chảy tới sink nguy hiểm `system()` – đây là lỗi Command Injection đã học ở Chương 1.

Giới hạn của Static Analysis (nhắc lại và mở rộng)

- Không phát hiện được lỗi phụ thuộc *giá trị dữ liệu cụ thể tại runtime* một cách chính xác tuyệt đối (vd: race condition phụ thuộc thời điểm lập lịch hệ điều hành)
- Alias analysis (xác định 2 con trỏ có trỏ tới cùng vùng nhớ hay không) là bài toán rất khó, ảnh hưởng độ chính xác của mọi kỹ thuật static analysis dựa trên con trỏ
- Đây là động lực chuyển sang **Bounded Model Checking** với SAT/SMT solver (Chương 3) – kỹ thuật có độ chính xác cao hơn cho một phạm vi giới hạn

Bài tập: Phân loại lỗi từ output công cụ

Một công cụ static analysis báo: *”Warning: variable ’x’ may be used uninitialized”* tại dòng chứa `if (cond) x = 5; return x + 1;`
Đây có phải luôn là lỗi thật không? Giải thích khả năng false positive.

Đáp án: Phân loại lỗi từ output công cụ

- Đây **có thể là false positive** nếu lập trình viên (qua logic ở nơi khác) đảm bảo cond luôn đúng trước khi tới đây – nhưng công cụ static analysis không ”biết” điều này nếu logic quá phức tạp để suy luận
- Tuy nhiên đây cũng **có thể là lỗi thật** nếu thực sự tồn tại đường thực thi mà cond sai
- Cách xử lý đúng: luôn khởi tạo x tường minh (`int x = 0;`) để loại bỏ hoàn toàn nguy cơ, dù công cụ có báo đúng hay không

Bảng so sánh công cụ đã học

Công cụ	Kỹ thuật	Tốc độ	Độ sâu phân tích
cppcheck	Pattern-based	Rất nhanh	Nông
clang-tidy	Rule-based + một phần data-flow	Nhanh	Trung bình
Clang Static Analyzer	Symbolic execution	Chậm hơn	Sâu
Valgrind (Ch2.2)	Dynamic (runtime)	Chậm	Rất chính xác (runtime)

Abstract Interpretation – Ví dụ trực quan

Định nghĩa

Abstract Interpretation: thay vì theo dõi giá trị *chính xác* của biến, công cụ theo dõi một *miền trừu tượng* (vd: khoảng giá trị $[min, max]$) an toàn bao trùm mọi giá trị có thể có.

Ví dụ

Với `int x = input(); if (x > 0) x = x - 1;`, công cụ suy luận: trước if, $x \in [-\infty, \infty]$; trong nhánh true, $x \in [1, \infty]$; sau phép trừ, $x \in [0, \infty]$ – không cần biết giá trị chính xác vẫn chứng minh được $x \geq 0$ tại điểm đó.

Công cụ công nghiệp: Astrée

- Công cụ static analysis dựa trên abstract interpretation, phát triển cho ngành hàng không (Airbus)
- Có thể chứng minh **không có** runtime error (chia 0, tràn số, truy cập ngoài mảng) cho một chương trình C cụ thể một cách *sound* (không bỏ sót)
- Đánh đổi: chi phí phân tích rất cao, chỉ khả thi cho mã nguồn quan trọng (safety-critical), không phù hợp chạy hàng ngày trên codebase lớn thông thường

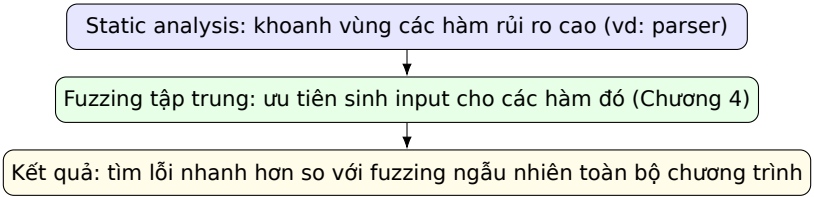
Syntax-based vs. Semantic-based Analysis

Syntax-based	Chỉ dựa trên cấu trúc cú pháp mã nguồn (vd: tìm chuỗi `"strcpy("` – nhanh nhưng dễ nhầm (vd: `strcpy` nằm trong comment hoặc string literal)
Semantic-based	Hiểu ý nghĩa thực sự của mã nguồn qua AST/CFG, phân biệt được ngữ cảnh – chậm hơn nhưng chính xác hơn nhiều

Bug Pattern Database (CWE Checkers)

- Nhiều công cụ static analysis được xây dựng dựa trên cơ sở dữ liệu các *mẫu lỗi đã biết* tương ứng với từng mã CWE (đã học ở Chương 1)
- Ví dụ: một ”checker” riêng cho CWE-120 (Buffer Overflow), một checker khác cho CWE-416 (Use After Free)
- Ưu điểm: dễ giải thích kết quả (biết chính xác đang kiểm tra loại lỗi nào); Nhược điểm: chỉ phát hiện được lỗi *đã biết mẫu*, có thể bỏ sót lỗi logic mới lạ

Kết hợp Static Analysis và Fuzzing



Đây là chiến lược thực tế phổ biến trong kiểm thử bảo mật công nghiệp: dùng static analysis để *định hướng* nỗ lực fuzzing, thay vì áp dụng độc lập.

Bài tập: Chọn công cụ phù hợp theo tình huống

Với mỗi tình huống, chọn công cụ phù hợp nhất (cppcheck, clang-tidy, Clang Static Analyzer, Valgrind):

1. Cần rà soát nhanh hàng nghìn file C trước mỗi lần commit
2. Nghi ngờ có use-after-free phức tạp qua nhiều lời gọi hàm, cần phân tích sâu dù chậm hơn
3. Đã có chương trình chạy được, muốn xác nhận không có memory leak khi chạy thực tế

Đáp án: Chọn công cụ phù hợp theo tình huống

1. **cppcheck** hoặc **clang-tidy** – nhanh, phù hợp chạy tự động trong CI cho khối lượng lớn
2. **Clang Static Analyzer** – symbolic execution giúp theo dõi lỗi qua nhiều bước gọi hàm
3. **Valgrind** – công cụ dynamic, xác nhận hành vi thực tế khi chương trình chạy

Đọc hiểu báo cáo Static Analyzer: Path-sensitive

```
warning: Use of memory after it is freed
  note: Memory is allocated at line 3
  note: Memory is freed at line 7
  note: Use happens here at line 9
```

Báo cáo *path-sensitive* (như của Clang Static Analyzer) chỉ ra **toàn bộ đường đi** dẫn tới lỗi (alloc $\rightarrow$ free $\rightarrow$ use), giúp lập trình viên hiểu ngay nguyên nhân thay vì chỉ biết dòng lỗi cuối cùng.

Chi phí vận hành Static Analysis trong tổ chức thực tế

- Chạy full-scan mỗi commit có thể quá chậm với codebase hàng triệu dòng → thường chỉ scan phần thay đổi (incremental analysis)
- Cần quy trình *triage* (phân loại) cảnh báo: gán người phụ trách, đánh dấu false positive đã xác nhận để công cụ không báo lại
- Đo lường hiệu quả: tỷ lệ lỗi thật/tổng cảnh báo (precision), tỷ lệ lỗi thật được tìm thấy/tổng lỗi thật tồn tại (recall) – áp dụng khái niệm tương tự precision/recall trong học máy

Custom Static Analysis Rules – Ý tưởng nâng cao

- Các tổ chức lớn thường viết *checker riêng* cho lỗi đặc thù của codebase mình (vd: quy tắc gọi API nội bộ đúng thứ tự)
- Công cụ như `clang-tidy` và Clang Static Analyzer hỗ trợ plugin để mở rộng, không giới hạn ở các checker có sẵn
- Đây là bước chuyển tiếp tự nhiên sang việc *tự viết công cụ phân tích tĩnh* – chính là nội dung module Python `ch2_memsafe.pattern_scanner` sinh viên sẽ thực hành

Bài tập: Viết Rule tùy chỉnh (khái niệm)

Giả sử tổ chức của bạn cấm dùng hàm `system()` vì rủi ro command injection (đã học ở Chương 1). Hãy mô tả (bằng lời, không cần code) cách một static analysis rule đơn giản có thể phát hiện vi phạm này.

Đáp án: Viết Rule tùy chỉnh

- Rule đơn giản nhất (syntax-based): quét mã nguồn tìm chuỗi `"system("`, báo cáo mọi vị trí xuất hiện kèm số dòng
- Cải tiến: loại trừ các trường hợp nằm trong comment hoặc string literal (cần phân tích cú pháp thay vì chỉ tìm chuỗi thô)
- Đây chính là logic cốt lõi sẽ cài đặt trong `pattern_scanner.py` của module Python

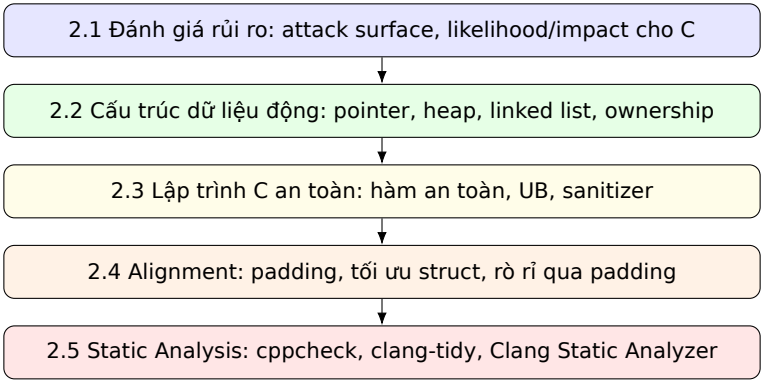
Static Analysis cho Build System lớn: Thách thức thực tế

- Với codebase hàng triệu dòng (vd: nhân Linux), full scan có thể mất hàng giờ
- Giải pháp: phân tích gia tăng (incremental analysis) – chỉ scan file/hàm bị thay đổi trong mỗi commit
- Kết hợp với hệ thống theo dõi cảnh báo đã ”chấp nhận” (baseline suppression) để tránh lặp lại cảnh báo cũ đã được xác nhận false positive

Tổng kết 2.5

- Static analysis gồm 3 nhóm kỹ thuật: pattern-based, data-flow, abstract interpretation
- Công cụ thực tế: cppcheck (nhanh, nông), clang-tidy (coding standard), Clang Static Analyzer (sâu, symbolic execution)
- False positive/negative là đánh đổi cố hữu – cần kết hợp nhiều công cụ và kỹ thuật khác (dynamic, fuzzing)
- Tích hợp CI/CD giúp phát hiện lỗi sớm, giảm chi phí sửa lỗi

Bản đồ tri thức Chương 2



Bảng thuật ngữ cốt lõi Chương 2 (1/2)

Attack Surface	Tổng hợp các điểm dữ liệu ngoài có thể đi vào chương trình
Ownership	Quy ước rõ ràng ai chịu trách nhiệm free một vùng nhớ
Undefined Behavior	Hành vi chuẩn C không quy định kết quả, trình biên dịch có thể tối ưu theo cách gây lỗi
Padding	Byte đệm trình biên dịch thêm vào struct để đảm bảo alignment

Bảng thuật ngữ cốt lõi Chương 2 (2/2)

ASan/UBSan	Công cụ dynamic phát hiện lỗi bộ nhớ/undefined behavior khi chạy
Valgrind	Công cụ dynamic phân tích bộ nhớ chi tiết, chi phí runtime cao
False Positive/Negative	Cảnh báo sai có lỗi / bỏ sót lỗi thật của công cụ phân tích
Abstract Interpretation	Kỹ thuật static analysis tính xấp xỉ an toàn của mọi trạng thái chương trình

Luyện tập trắc nghiệm – Câu 1–3

Câu 1. Vì sao `struct {char a; int b;}` thường có kích thước 8 byte thay vì 5?
A. Lỗi trình biên dịch B. Padding để căn chỉnh b C. Vì char chiếm 4 byte D. Do hệ điều hành quy định
Câu 2. Công cụ nào dùng kỹ thuật symbolic execution để phân tích mã C?
A. cppcheck B. clang-tidy C. Clang Static Analyzer D. gdb
Câu 3. False negative trong static analysis nghĩa là gì?
A. Báo lỗi sai B. Bỏ sót lỗi thật C. Chương trình chạy chậm D. Không biên dịch được

Đáp án Câu 1–3

1. B (Padding để căn chỉnh b tại địa chỉ chia hết cho 4).
2. C (Clang Static Analyzer).
3. B (Bỏ sót lỗi thật – nguy hiểm hơn false positive).

Luyện tập trắc nghiệm – Câu 4–6

Câu 4. Vì sao cần `memset` struct về 0 trước khi gửi qua mạng?
A. Tăng tốc độ gửi B. Tránh rò rỉ dữ liệu qua padding chưa khởi tạo C. Bắt buộc theo chuẩn C D. Giảm kích thước struct
Câu 5. Đâu là ví dụ về Undefined Behavior trong C?
A. Chia cho 0 với số nguyên B. In ra màn hình C. Khai báo biến D. Gọi hàm
Câu 6. `#pragma pack(1)` có tác dụng gì?
A. Tăng tốc truy cập B. Loại bỏ padding C. Tăng kích thước struct D. Không có tác dụng

Đáp án Câu 4–6

4. B (Tránh rò rỉ dữ liệu cũ còn sót trong padding).
5. A (Chia cho 0 với số nguyên là UB trong C).
6. B (Loại bỏ padding, có thể đánh đổi hiệu năng/tính di động).

Bài tập tổng hợp: Rà soát một module C hoàn chỉnh

Cho một module quản lý danh sách sinh viên (linked list of struct Student), có chức năng: thêm sinh viên, xóa theo ID, ghi toàn bộ danh sách ra file nhị phân.

1. Đề xuất cấu trúc `struct Student` tối ưu alignment
2. Liệt kê 3 rủi ro bộ nhớ tiềm ẩn trong 3 chức năng trên
3. Đề xuất công cụ (static/dynamic) phù hợp để kiểm tra từng loại rủi ro

Gợi ý đáp án bài tập tổng hợp

- Struct: sắp xếp trường lớn (double, long) trước, trường nhỏ (char) sau
- Rủi ro: (1) memory leak khi xóa sinh viên nếu quên free node; (2) rò rỉ padding khi ghi struct ra file nhị phân nếu không memset; (3) use-after-free nếu duyệt tiếp sau khi free node hiện tại
- Công cụ: Valgrind/ASan cho memory leak và use-after-free (dynamic); cppcheck/clang-tidy để rà soát pattern nguy hiểm sớm (static); review thủ công cho vấn đề padding/memset

Cập nhật dự án Python: module ch2_memsafe

Module `ch2_memsafe` trong `softsec-toolkit` gồm:

- `pattern_scanner.py`: quét mã nguồn C tìm hàm nguy hiểm (`strcpy`, `gets`, `sprintf`, `strcat`...), báo dòng và mức độ nghiêm trọng
- `memory_trace.py`: mô phỏng "vết thực thi" gồm chuỗi lệnh `alloc`/`free`/`access`, phát hiện tự động double-free, use-after-free, memory leak
- `struct_layout.py`: tính toán kích thước và padding của struct từ danh sách trường (áp dụng kiến thức mục 2.4)

Demo module: pattern_scanner

```
from softsec.ch2_memsafe import scan_source

findings = scan_source(open("vulnerable.c").read())
for f in findings:
    print(f"Line {f.line}: {f.function} - {f.severity}")
# Line 3: strcpy - HIGH
# Line 7: gets - CRITICAL
```

Demo module: memory_trace

```python
from softsec.ch2_memsafe import MemoryTrace

trace = MemoryTrace()
trace.alloc("p1", size=64)
trace.free("p1")
trace.access("p1")   # Phat hien: USE-AFTER-FREE!
print(trace.get_violations())
```

Checklist tự đánh giá cuối chương

- ☐ Tôi hiểu vì sao struct có padding và có thể tính toán kích thước struct
- ☐ Tôi phân biệt được stack, heap và các lỗi bộ nhớ động phổ biến
- ☐ Tôi biết ít nhất 2 công cụ static analysis và 2 công cụ dynamic analysis cho C
- ☐ Tôi có thể rà soát một đoạn mã C ngắn và chỉ ra lỗi bộ nhớ tiềm ẩn

Danh sách kỹ năng thực hành đã đạt được (Chương 2)

Sau Chương 2, sinh viên đã có thể:

- Tính toán kích thước struct với padding và tối ưu hóa thứ tự trường
- Sử dụng cppcheck, `clang-tidy`, biên dịch với ASan/UBSan
- Nhận diện và sửa các lỗi bộ nhớ phổ biến trong mã C: buffer overflow, use-after-free, double free, memory leak
- Đọc hiểu báo cáo path-sensitive từ công cụ static analysis nâng cao

So sánh nhanh: Chương 1 vs Chương 2

	Chương 1	**Chương 2**
Phạm vi	Khái niệm tổng quát, đa ngôn ngữ	Cụ thể hóa cho C
Mô hình	Kripke structure (trừu tượng)	Memory model thực tế (heap/stack)
Công cụ	Giới thiệu khái niệm	Thực hành cppcheck, ASan, Valgrind

Điểm cần lưu ý khi ôn tập Chương 2

- Phân biệt rõ các loại lỗi bộ nhớ và triệu chứng của từng loại (không nhầm use-after-free với double-free)
- Nhớ công thức tính alignment: kích thước struct luôn là bội số của alignment lớn nhất
- Phân biệt rõ static analysis (compile-time) và dynamic analysis (runtime, cần chạy chương trình)

Câu hỏi mở để suy ngẫm

Nếu ngôn ngữ C không có `malloc`/`free` thủ công mà có garbage collector tự động như Java, những lớp lỗi nào đã học trong Chương 2 sẽ *biến mất hoàn toàn*? Những lớp lỗi nào *vẫn còn tồn tại*?

Gợi ý trả lời: Câu hỏi mở

- **Biến mất**: memory leak (GC tự thu hồi), use-after-free, double-free (GC không giải phóng khi vẫn còn tham chiếu)
- **Vẫn còn tồn tại**: buffer overflow logic (truy cập sai chỉ số mảng), race condition, business logic error, injection (nếu ngôn ngữ vẫn tương tác với hệ thống/CSDL bên ngoài)
- Bài học: garbage collector giải quyết *một lớp* vấn đề bộ nhớ, nhưng không phải *mọi* vấn đề an toàn phần mềm

Kết nối tới Chương 3

Chương 3 – Phát hiện lỗ hổng phần mềm: Phân tích tĩnh sẽ đi sâu hơn vào:

- 3.1–3.3: Nền tảng verification/validation, soundness/completeness, so sánh static/dynamic/debug
- 3.4 SAT/SMT Solving – nền tảng toán học cho phân tích tĩnh nâng cao
- 3.5 Bounded Model Checking – áp dụng Kripke structure (Chương 1) vào kiểm chứng mã thực tế
- 3.6–3.7 Memory Model và Aligned Memory Model – hình thức hóa các khái niệm alignment đã học ở mục 2.4

Câu hỏi ôn tập tổng hợp (1/3)

1. Vì sao sắp xếp lại thứ tự trường trong struct có thể giảm kích thước mà không mất dữ liệu?
2. Kể tên 3 công cụ static analysis đã học và điểm mạnh/yếu của mỗi công cụ
3. Vì sao `union` tiềm ẩn rủi ro về type punning?

Câu hỏi ôn tập tổng hợp (2/3)

4. False sharing khác gì với race condition (đã học ở Chương 1)?
5. Precision và Recall trong ngữ cảnh static analysis nghĩa là gì?
6. Vì sao cần `memset` struct trước khi ghi ra file/mạng?

Câu hỏi ôn tập tổng hợp (3/3)

7. Abstract interpretation khác gì với kiểm tra giá trị cụ thể (concrete execution)?
8. Vì sao kết hợp static analysis với fuzzing hiệu quả hơn dùng riêng lẻ?
9. Path-sensitive analysis mang lại lợi ích gì so với chỉ báo dòng lỗi đơn lẻ?

Đáp án gợi ý câu 1–3

1. Padding chỉ phụ thuộc *vị trí tương đối* giữa các trường theo yêu cầu alignment, không phụ thuộc thứ tự khai báo cụ thể – nhóm các trường alignment lớn lại với nhau giảm số lần ”làm tròn” padding cần thiết
2. cppcheck: nhanh, nông, dễ dùng; clang-tidy: tập trung coding standard, tích hợp IDE tốt; Clang Static Analyzer: sâu (symbolic execution) nhưng chậm hơn
3. Vì các trường của union *chia sẻ cùng vùng nhớ vật lý* – ghi qua kiểu này rồi đọc qua kiểu khác có thể cho kết quả phụ thuộc kiến trúc/trình biên dịch (không portable, có thể là UB)

Đáp án gợi ý câu 4–6

4. False sharing là vấn đề *hiệu năng* (cache coherence) dù dữ liệu logic không xung đột; race condition là vấn đề *đúng đắn* (kết quả sai) khi 2 luồng truy cập cùng dữ liệu logic không đồng bộ
5. Precision = tỷ lệ cảnh báo đúng thực sự trong tổng số cảnh báo; Recall = tỷ lệ lỗi thật được phát hiện trong tổng số lỗi thật tồn tại
6. Vì byte padding chưa khởi tạo có thể chứa dữ liệu ngẫu nhiên còn sót từ bộ nhớ trước đó, dẫn tới rò rỉ thông tin nếu truyền ra ngoài chương trình

Đáp án gợi ý câu 7–9

7. Abstract interpretation xét *mọi giá trị có thể có* cùng lúc qua miền trừu tượng (vd: khoảng giá trị), trong khi concrete execution chỉ xét một giá trị cụ thể tại một thời điểm
8. Static analysis khoanh vùng khu vực rủi ro cao để fuzzing tập trung nỗ lực, tăng hiệu quả tìm lỗi so với fuzzing ngẫu nhiên đều khắp chương trình
9. Path-sensitive analysis cho biết *toàn bộ chuỗi sự kiện* dẫn tới lỗi (alloc $\rightarrow$ free $\rightarrow$ use), giúp lập trình viên hiểu và sửa lỗi nhanh hơn nhiều so với chỉ biết dòng lỗi cuối

Luyện tập trắc nghiệm – Câu 7–9

Câu 7. Đâu là công cụ dynamic analysis (không phải static)?
A. cppcheck B. clang-tidy C. Valgrind D. Clang Static Analyzer
Câu 8. `struct Point { int x, y; }` có alignment bằng bao nhiêu (giả sử int = 4 byte)?
A. 1 B. 2 C. 4 D. 8
Câu 9. Kỹ thuật nào chứng minh *sound* rằng chương trình không có runtime error?
A. Fuzzing ngẫu nhiên B. Code review thủ công C. Abstract Interpretation (vd: Astrée) D. Chạy unit test

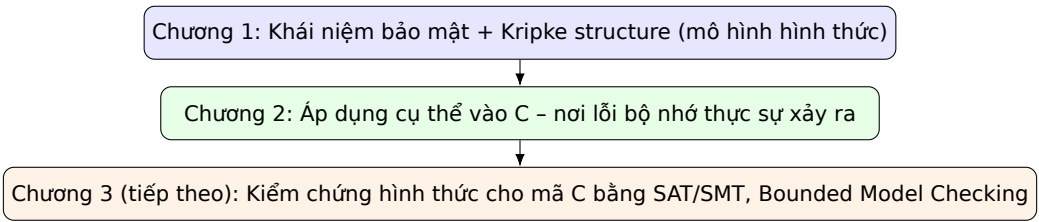
Đáp án Câu 7–9

7. C (Valgrind – công cụ dynamic, cần chạy chương trình).
8. C (4 – bằng alignment của trường `int` lớn nhất).
9. C (Abstract Interpretation, vd: Astrée – có thể sound cho phạm vi runtime error xác định).

Bảng công thức và quy tắc cần nhớ

Khái niệm	Quy tắc/Công thức
Kích thước struct	Bội số của alignment lớn nhất trong các trường
Precision	(Cảnh báo đúng) / (Tổng cảnh báo)
Recall	(Lỗi thật tìm thấy) / (Tổng lỗi thật)
Union size	max(kích thước các trường)

Liên hệ tổng thể: Chương 1 + Chương 2



Bài tập thực hành mở rộng

Viết một chương trình C nhỏ (khoảng 50 dòng) mô phỏng quản lý một danh sách sản phẩm (thêm/xóa/tìm kiếm bằng linked list). Sau đó:

1. Chạy cppcheck và sửa mọi cảnh báo mức `error`
2. Biên dịch với `-fsanitize=address` và chạy thử với dữ liệu biên (danh sách rỗng, xóa phần tử không tồn tại)
3. Tính toán và tối ưu kích thước struct sản phẩm theo alignment

Vai trò Chương 2 trong bài tập lớn cuối kỳ

- Nếu chọn đề tài liên quan tới mã C, sinh viên cần áp dụng toàn bộ quy trình: đánh giá rủi ro (2.1) $\rightarrow$ rà soát cấu trúc dữ liệu (2.2) $\rightarrow$ áp dụng coding an toàn (2.3) $\rightarrow$ tối ưu alignment nếu liên quan hiệu năng (2.4) $\rightarrow$ chạy công cụ static analysis (2.5) và báo cáo kết quả
- Module Python `ch2_memsafe` là công cụ hỗ trợ trực tiếp cho quy trình này

Tài liệu tham khảo thêm cho Phần 2

- ”What Every Programmer Should Know About Memory” (Ulrich Drepper) – chi tiết về alignment, cache
- Cppcheck Manual, Clang Static Analyzer documentation
- CERT C Coding Standard – các quy tắc MEM, STR, INT đầy đủ
- Astrée Static Analyzer – tài liệu kỹ thuật về abstract interpretation

Kết nối tới bài kiểm tra giữa kỳ

- Kiểm tra giữa kỳ (trắc nghiệm) sẽ bao phủ *toàn bộ Chương 1 và Chương 2*
- Ưu tiên ôn tập: công thức tính kích thước struct, phân biệt các loại lỗi bộ nhớ, phân biệt công cụ static/dynamic, và các khái niệm Kripke structure/risk assessment
- Luyện lại toàn bộ câu hỏi trắc nghiệm có đáp án trong cả 4 phần đã học (Chương 1 Phần 1-2, Chương 2 Phần 1-2)

Hỏi đáp

Câu hỏi?

Chuẩn bị dự án Python trước Chương 3

Trước khi sang Chương 3, module `ch2_memsafe` cần hoàn thiện với đầy đủ unit test (tương tự `ch1_models`), bao gồm test cho `pattern_scanner`, `memory_trace`, và `struct_layout`. Đây sẽ là nền tảng để Chương 3 xây dựng `ch3_staticsat` kết hợp cả hai.

Thảo luận mở rộng cuối chương

- Trong thực tế, một dự án C lớn (vd: OpenSSL, nginx) áp dụng *tất cả* các kỹ thuật đã học: coding standard, static analysis CI, sanitizer trong test, và cả kiểm chứng hình thức cho các phần lõi quan trọng nhất
- Không có một kỹ thuật ”hoàn hảo” duy nhất – an toàn phần mềm thực chất là chiến lược **phòng thủ nhiều lớp**, đúng như nguyên lý defense in depth đã học ở Chương 1

Kết thúc Chương 2

Hoàn thành Chương 2: Lập trình C an toàn – Quản lý bộ nhớ

Hẹn gặp lại ở Chương 3: Phát hiện lỗ hổng phần mềm – Phân tích tĩnh

Bài tập cuối cùng: Tự đánh giá toàn diện

Hãy tự chấm điểm hiểu biết của mình (1-5) cho từng mục, và ôn lại các mục dưới 4 điểm:

- Đánh giá rủi ro cho mã C (2.1)
- Quản lý cấu trúc dữ liệu động an toàn (2.2)
- Lập trình C an toàn và công cụ sanitizer (2.3)
- Alignment và tối ưu struct (2.4)
- Sử dụng và diễn giải công cụ static analysis (2.5)

Ghi chú dành cho bài tập lớn

Nếu bài tập lớn của bạn liên quan đến mã C, hãy đảm bảo báo cáo trình bày rõ: (1) mô hình rủi ro ban đầu, (2) các lỗi cụ thể tìm thấy kèm số dòng, (3) công cụ đã dùng để phát hiện, (4) bản vá đề xuất, và (5) xác nhận lại bằng cách chạy lại công cụ sau khi sửa.

Nguồn thực hành thêm

- OWASP Benchmark – bộ mã nguồn có chứa lỗi cố ý để luyện tập static analysis
- Juliet Test Suite (NIST) – hàng nghìn ví dụ mã C/C++ theo từng CWE cụ thể
- Google OSS-Fuzz – ví dụ thực tế về kết hợp static analysis và fuzzing quy mô lớn

Chuyển tiếp

Tiếp theo: Chương 3 – Phát hiện lỗ hổng phần mềm: Phân tích tĩnh